1.a) $f(z) = \frac{1}{2n}\|r - zV\|^2 + \lambda(1-\alpha)\frac{z^2}{2} + \lambda\alpha|z|$

$$= \frac{1}{2n}(r - zV)^T(r - zV) + \lambda(1-\alpha)\frac{z^2}{2} + \lambda\alpha|z|$$

$$= \frac{1}{2n}\left(\|r\|^2 - 2z(r^TV) + z^2\|V\|^2\right) + \frac{\lambda(1-\alpha)z^2}{2} + \lambda\alpha|z|$$

$$= \frac{\|r\|^2}{2n} - \frac{z(r^TV)}{n} + \frac{z^2\|V\|^2}{2n} + \frac{\lambda(1-\alpha)}{2}z^2 + \lambda\alpha|z|$$

$$= z^2\left(\frac{\|V\|^2 + \lambda n(1-\alpha)}{2n}\right) - z\left(\frac{r^TV}{n}\right) + \lambda|\alpha|z| + \underbrace{\frac{\|r\|^2}{2n}}_{\text{drop constants}}$$

Using the method in class:

$$= z^2 - z\left(\frac{r^TV}{\cancel{n}}\right)\left(\frac{2\cancel{n}}{\|V\|^2 + \lambda n(1-\alpha)}\right) + \frac{2n\lambda|\alpha|}{\|V\|^2 + \lambda n(1-\alpha)}|z|$$

$$= z^2 - 2z\left(\frac{r^TV}{\|V\|^2 + \lambda n(1-\alpha)}\right) + \frac{2n\lambda|\alpha|}{\|V\|^2 + \lambda n(1-\alpha)}|z|$$

$$\min_{x\in\mathbb{R}}\left[x^2 - 2x\hat{\beta}_j^{LS} + \lambda|x|\right] \Rightarrow \operatorname{sign}(\hat{\beta}_j^{LS})\left(|\hat{\beta}_j^{LS}| - \frac{\lambda}{2}\right)\mathbb{1}\left\{|\hat{\beta}_j^{LS}| > \frac{\lambda}{2}\right\}$$

$$\therefore \text{Soft}_{\frac{\lambda}{2}} \Rightarrow \text{Soft}_{\frac{n\lambda|\alpha|}{\|V\|^2 + \lambda n(1-\alpha)}}\left(\frac{r^TV}{\|V\|^2 + n\lambda(1-\alpha)}\right)$$

457_a3

2026-03-04

Question 1 b)

```r
library(glmnet)
```

```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-10
```

```r
soft_threshold <- function(u, lambda) {
  # Input: scalar u, positive scalar lambda
  # Output: if |u| <= lambda, output 0; if u > lambda, output u-lambda;
  # if u < -lambda, output u + lambda
  return(sign(u) * max(0, abs(u) - lambda))
}

# --- Coordinate descent for Elastic Net ---
# Modified from myLasso to include the alpha parameter and Elastic Net update rule
myElasticNet <- function(X, y, lambda, alpha, tol = 1e-16, maxitr = 1000) {
  n <- nrow(X)
  p <- ncol(X)
  beta = rep(0, p) # initialization

  for (k in 1:maxitr) { # the grand loop
    beta_old = beta # record the solution from previous iteration

    for (j in 1:p) { # update each beta_j in a loop
      # first, we calculate the partial residual by removing the effect of others
      r = y - X[, -j] %*% beta[-j]

      # Derived components for Elastic Net update:
      denom = sum(X[, j]^2) + n * lambda * (1 - alpha)
      target = (t(X[, j]) %*% r) / denom
      threshold = (n * lambda * alpha) / denom

      # update beta[j] using the soft-thresholding operator
      beta[j] <- soft_threshold(target, threshold)
    }

    # check if beta changed more than the tolerance level in this iteration
    if (sum(abs(beta - beta_old)) < tol) break
  }
```

```r
  return(list("itr" = k, "beta" = beta))
}

# --- Test correctness of the implementation ---
X <- as.matrix(read.csv("Q1A3_X.csv"))
y <- as.matrix(read.csv("Q1A3_Y.csv"))

lambda_val = 0.1
alpha_val = 0.6

# (1) beta_hat_1 produced by my algorithm
myfit = myElasticNet(X, y, lambda = lambda_val, alpha = alpha_val)
beta_hat_1 = myfit$beta
print(beta_hat_1)
```

```
##  [1]  0.5314323 -0.3028907  0.1510586  0.0000000  0.0000000  0.0000000
##  [7]  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000
## [13]  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000
## [19]  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000
## [25]  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000
```

```r
# (2) beta_hat_2 produced by the "glmnet" function
lassofit <- glmnet(X, y, alpha = alpha_val, lambda = lambda_val, intercept = FALSE)
beta_hat_2 = as.vector(lassofit$beta)
print(beta_hat_2)
```

```
##  [1]  0.5314327 -0.3028907  0.1510586  0.0000000  0.0000000  0.0000000
##  [7]  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000
## [13]  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000
## [19]  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000
## [25]  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000  0.0000000
```

```r
# Reporting the difference
mse_diff = mean((beta_hat_1 - beta_hat_2)^2)
print(mse_diff)
```

```
## [1] 7.637897e-15
```

2a) A natural cubic spline with knots at $x_1, \dots, x_n$ is a function that:

i) is a cubic polynomial on each interval $[x_i, x_{i+1}]$

ii) twice differentiable at each knot and continuous

iii) is linear outside range $[x_1, x_n]$. $g''(x) = 0$ for $x \le x_1$ and $x \ge x_n$

Since we have n knots and n functions $g(x_i)$, and a natural cubic spline the set of natural cubic spline forms a linear space with dimension n.

We want $\tilde{g}(x_i) = g(x_i)$, $i = 1, \dots, n$.

These are n linear constraints on functions in an n-D. linear space.

$\hat{g}(x) = \sum_{i=1}^{n} \beta_k h_k(x)$. where h_k is a natural cubic spline basis

in matrix form: $F\beta = y$ where $F_{ik} = h_k(x_i)$, $y_i = g(x_i)$

The evaluation map of the knots is non-singular, so F is invertible

$\therefore$ $\beta = F^{-1} y$ is unique.

Conclusion: Because the space of NCS with n knots has dimension n,

we have n independent constraints

the linear system is invertible

so there must exists $\tilde{g}$ satisfying $\tilde{g}(x_i) = g(x_i)$, and it is unique.

b) Show $\int_a^b \tilde{g}''(x)\, h''(x)\, dx = 0$ where $h(x_i) = g(x_i) - \tilde{g}(x_i) = 0$.

$$= \tilde{g}''(x)\, h'(x) \Big|_a^b - \int_a^b \tilde{g}'''(x)\, h'(x)\, dx$$

because $\tilde{g}$ is a NCS

it is linear outside $[a,b]$

so $\tilde{g}''(a), \tilde{g}''(b) = 0$

$0 \cdot h'(b) - 0 \cdot h'(a) = 0$

$$= 0 - \int_a^b \tilde{g}'''(x)\, h'(x)\, dx$$

(next page ↓)

Because $[x_i, x_{i+1}]$ each interval $\tilde{g}$ is a cubic polynomial

$\tilde{g}'''(x)$ must be a constant number

let $c_i = g'''(x)$, $x \in [x_i, x_{i+1}]$

$$\int_a^b \tilde{g}'''(x)\, h'(x)\, dx$$

$$= \sum_{i=1}^{n-1} \int_{x_1}^{x_{i+1}} c_i\, h'(x)\, dx$$

$$= c_i \int_{x_i}^{x_{i+1}} h'(x)\, dx$$

$$= c_i \big(h(x_{i+1}) - h(x_i)\big)$$

but $h(x_i) = 0 \ \forall i$, so $h(x_{i+1}) - h(x_i) = 0$ $= 0$

$$\therefore \int_a^b \tilde{g}''(x)\, h''(x)\, dx = 0 - 0 = 0$$

c) Show $\int_a^b [g''(x)]^2\, dx = \int_a^b [\tilde{g}''(x)]^2\, dx + \int_a^b [h''(x)]^2\, dx$

because $h(x) = g(x) - \tilde{g}(x) \Rightarrow g(x) = \tilde{g}(x) + h(x)$

differentiate: $g''(x) = \tilde{g}''(x) + h''(x)$

square: $[g''(x)]^2 = [\tilde{g}''(x) + h''(x)]^2$

$$= (\tilde{g}'')^2 + 2\tilde{g}''h'' + (h'')^2$$

$$\int_a^b [g''(x)]^2 dx = \int_a^b [\tilde{g}''(x)]^2 dx + 2\int_a^b [\tilde{g}''h''] dx + \int_a^b (h'')^2 dx$$

but from b) $\int_a^b \tilde{g}''h''\, dx = 0$

so: $= \int_a^b [\tilde{g}''(x)]^2 + \int_a^b (h''(x))^2\, dx$

3 a) $$g_{\alpha,\beta}(x) = \sum_{j=1}^{d} \alpha_j \sin(j \times \pi \times x) + \beta_j \cos(j \times \pi \times x)$$

$$(\hat{\alpha}, \hat{\beta}) = \underset{\alpha, \beta}{\arg\min} \sum_{i=1}^{n} (y_i - g_{\alpha,\beta}(x_i))^2$$

Choose basis function $b_j(\cdot)$

$b_1(x) = \sin(1\pi x), \ldots, b_d(x) = \sin(d\pi x)$

$b_{d+1}(x) = \cos(1\pi x), \ldots b_{2d}(x) = \cos(d\pi x)$

Design matrix

$$X_{i,\cdot} = [\sin(1\pi x_i) \ldots \sin(d\pi x_i) \ \cos(1\pi x_i) \ \ldots \ \cos(d\pi x_i)]$$

This matrix should be of dimension $n \times 2d$, $1 \leq i \leq n$

Now our optimizer should be of dimension 2d, that looks like:

$$\Theta = [\alpha_1, \ldots, \alpha_d, \beta_1, \ldots, \beta_d]^T \qquad 20 \times 1$$

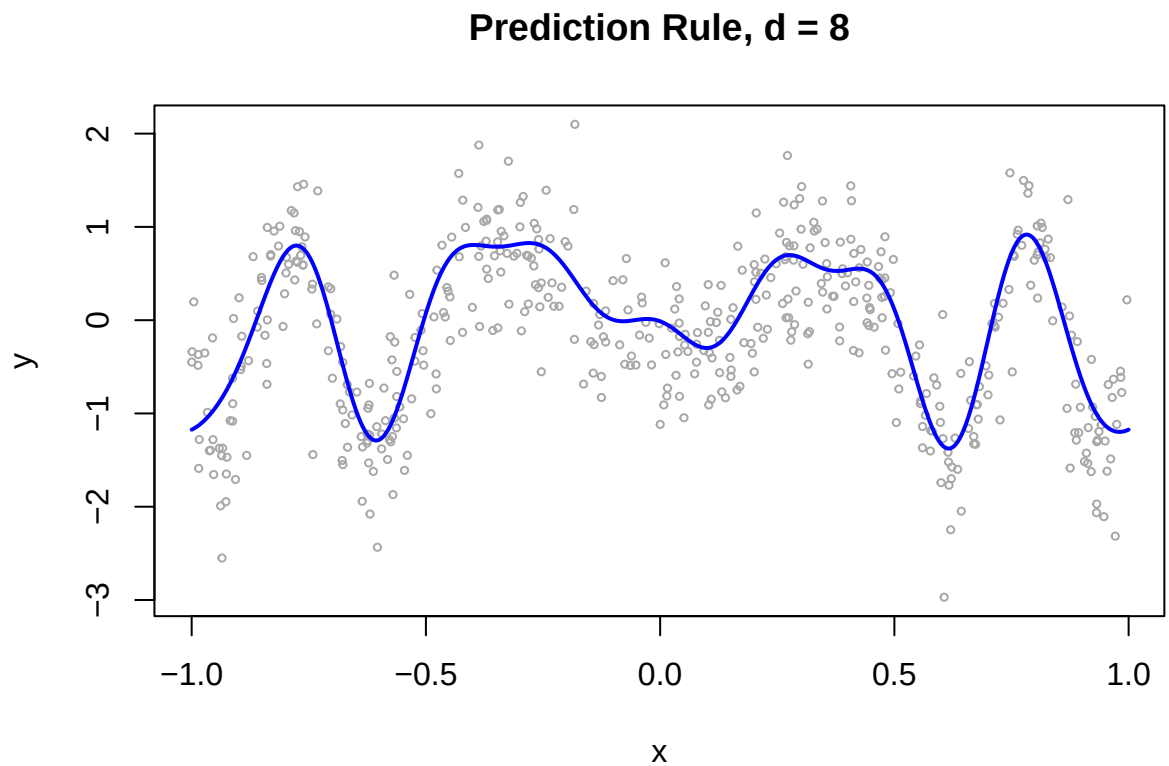
The optimization problem is thus defined as $\min_{\theta} \|Y - X\theta\|^2$

without an intercept.

And $\hat{\theta} = (X^T X)^{-1} X^T Y$ according to linear regression.

Question 3 b)

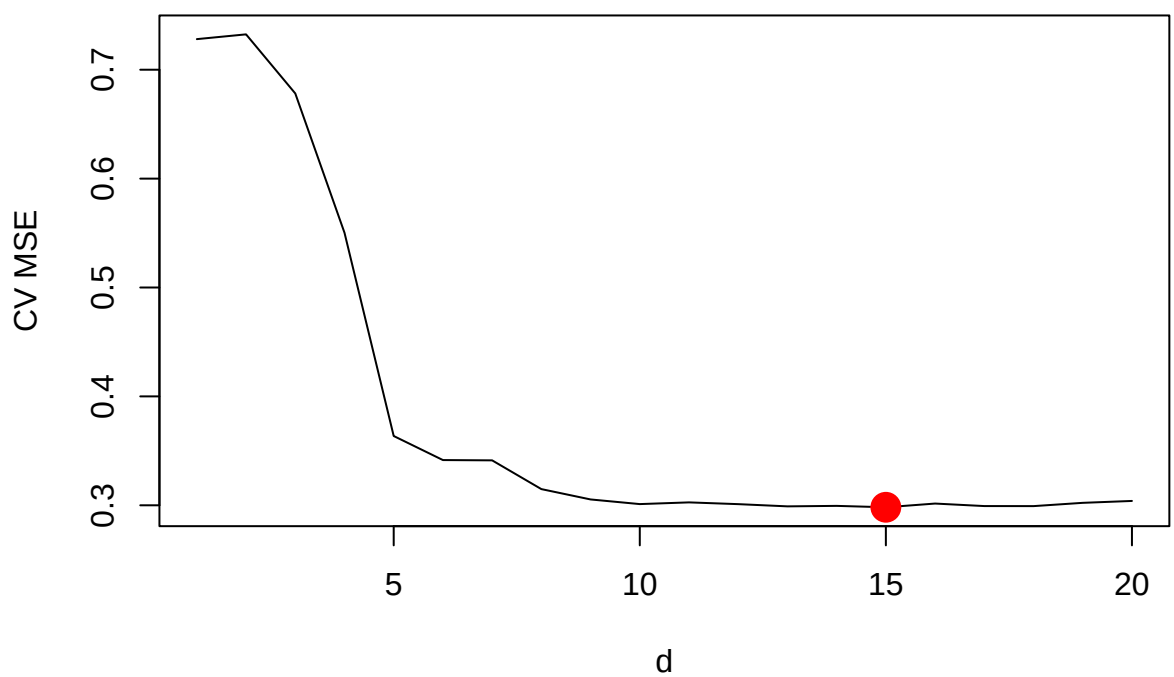
```r
library(boot)
train_data <- read.csv("Q4A3_train.csv")

# Split the dataset
x_train <- train_data[, 1]
y_train <- train_data[, 2]

# Basis Design Matrix
get_basis_matrix <- function(x_vec, d) {
  n_obs <- length(x_vec)
  X <- matrix(0, nrow = n_obs, ncol = 2 * d)
  for (j in 1:d) {
    X[, j] <- sin(j * pi * x_vec)      # Alpha basis
    X[, d + j] <- cos(j * pi * x_vec)  # Beta basis
  }
  return(X)
}

# d = 8 defined
d_fixed <- 8
X_train_8 <- get_basis_matrix(x_train, d_fixed)

# Normal Equation beta = (X'X)^-1 X'Y to solve linear regression
beta_hat_8 <- solve(t(X_train_8) %*% X_train_8) %*% t(X_train_8) %*% y_train

# Visualization on a dense grid
grid_points <- seq(-1, 1, length.out = 500)
X_grid_8 <- get_basis_matrix(grid_points, d_fixed)
preds_8 <- X_grid_8 %*% beta_hat_8

plot(x_train, y_train, col = "darkgrey", cex = 0.5, pch = 1,
     main = paste("Prediction Rule, d =", d_fixed),
     xlab = "x", ylab = "y")
lines(grid_points, preds_8, col = "blue", lwd = 2)
```



Question 3 c)

```
set.seed(13)

max_d <- 20
k <- 10
n <- length(x_train)

# Randomly assign each observation to a fold (1 to 10)
folds <- sample(rep(1:k, length.out = n))
cv_errors <- rep(0, max_d)

for (d in 1:max_d) {
  fold_mse <- rep(0, k)

  for (k in 1:k) {
    # Partition data into training and validation sets for this fold
    val_indices <- which(folds == k)

    x_train_fold <- x_train[-val_indices]
    y_train_fold <- y_train[-val_indices]
    x_val_fold   <- x_train[val_indices]
    y_val_fold   <- y_train[val_indices]

    # Construct matrices
    X_train_fold <- get_basis_matrix(x_train_fold, d)
    X_val_fold   <- get_basis_matrix(x_val_fold, d)
```

```r
    # Solve using Normal Equation
    beta_fold <- solve(t(X_train_fold) %*% X_train_fold) %*% t(X_train_fold) %*% y_train_fold

    # Predict and calculate MSE for this fold
    preds_val <- X_val_fold %*% beta_fold
    fold_mse[k] <- mean((y_val_fold - preds_val)^2)
  }

  # Average MSE across all 10 folds for current d
  cv_errors[d] <- mean(fold_mse)
}


plot(1:max_d, cv_errors, xlab = "d", ylab = "CV MSE", type = "l")
points(which.min(cv_errors), cv_errors[which.min(cv_errors)], col = "red", cex = 3, pch = 20)
```




```r
# Report CV results
print("Optimal d is:")
```

```
## [1] "Optimal d is:"
```

```r
(best_d_cv <- which.min(cv_errors))
```

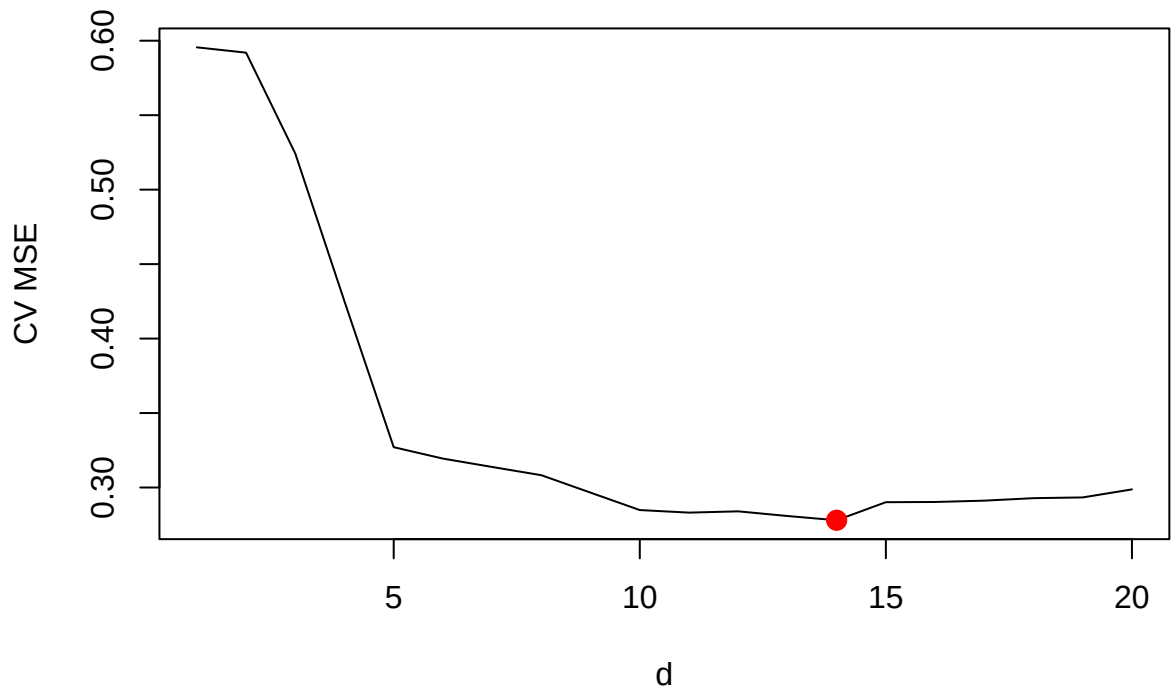
```
## [1] 15
```

```r
cv_errors
```

```
##  [1] 0.7280925 0.7324687 0.6782097 0.5502594 0.3636210 0.3415231 0.3412599
##  [8] 0.3148512 0.3053508 0.3011046 0.3026528 0.3010481 0.2990054 0.2994599
## [15] 0.2981869 0.3015785 0.2992770 0.2992311 0.3022433 0.3039716
```

Question 3 d)

```r
test_data  <- read.csv("Q4A3_test.csv")
x_test  <- test_data[, 1]
y_test  <- test_data[, 2]

test_errors <- rep(0, max_d)

for (d in 1:max_d) {
  # Train on the full training dataset
  X_tr <- get_basis_matrix(x_train, d)
  beta_full <- solve(t(X_tr) %*% X_tr) %*% t(X_tr) %*% y_train

  # Evaluate on the test dataset
  X_te <- get_basis_matrix(x_test, d)
  preds_te <- X_te %*% beta_full

  test_errors[d] <- mean((y_test - preds_te)^2)
}

plot(1:max_d, test_errors, xlab = "d", ylab = "CV MSE", type = "l")
points(which.min(test_errors), test_errors[which.min(test_errors)], col = "red", cex = 2, pch = 20)
```




```
test_errors
```

```
##  [1] 0.5955404 0.5919845 0.5242744 0.4249484 0.3270831 0.3194118 0.3137993
##  [8] 0.3082676 0.2965425 0.2848589 0.2831305 0.2840056 0.2808669 0.2780245
## [15] 0.2901467 0.2902785 0.2911713 0.2928475 0.2933410 0.2987416
```